Your Laptop Kubernetes Guide

Java microservices on Docker Desktop, explained in easy words

Inside

What is in this guide:

- What the project is and what each part does
- The technical details: tools, ports, databases, images
- Kubernetes words explained simply
- How the whole setup was done, step by step
- **Human steps vs Claude's steps**, and how Claude worked internally
- How to use it every day, all browser links, and how to fix problems

Laptop	lenovo-shikhin (Windows, 16 GB RAM)
Project	spring-boot-3-microservices-course (Spring Boot 3.2.4, Java 21)
Runs on	Docker Desktop 4.94, built-in Kubernetes v1.36 (Kubeadm, 1 node)
One script	Desktop\kubernetes\k8s.ps1
Written	8 October 2026

Use the bookmarks panel in your PDF viewer to jump to any section.

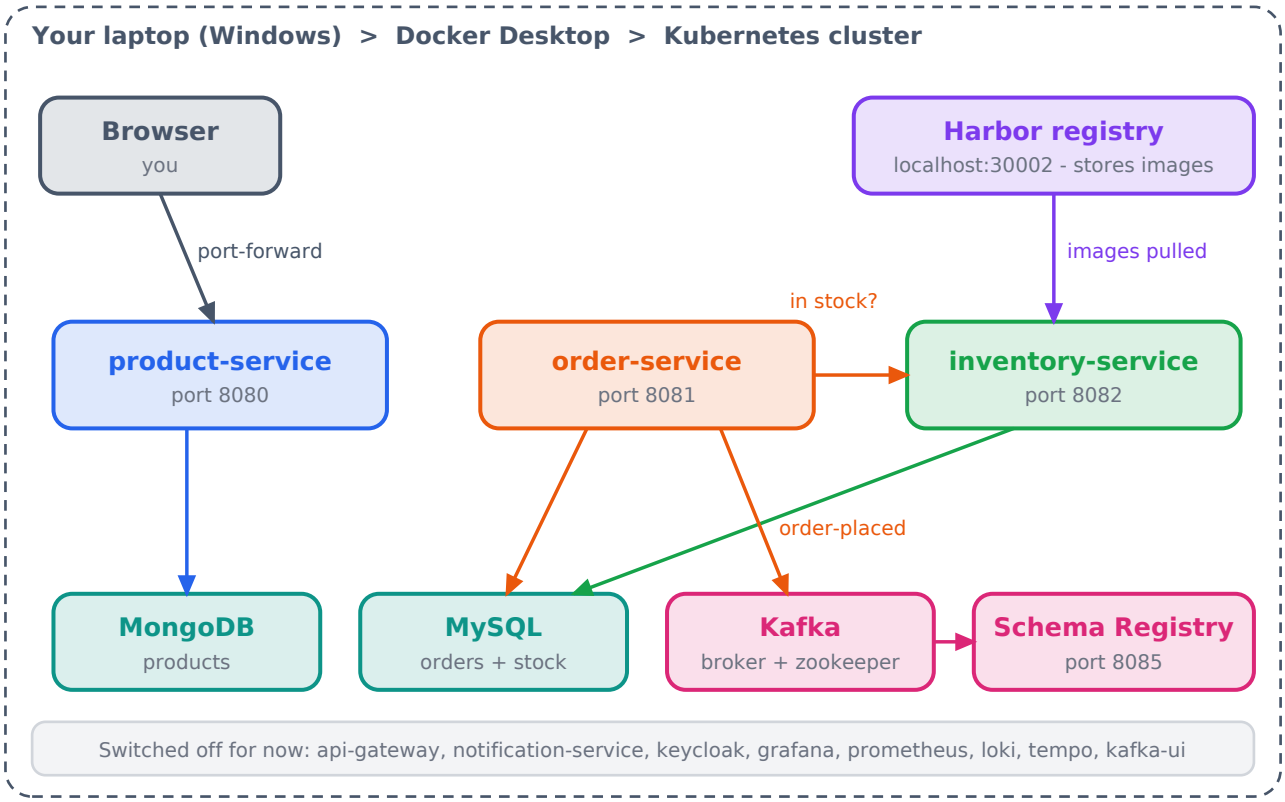
1 The big picture

What this project is, in one page

Think of the project as a **tiny online phone shop**. Instead of one big program, it is split into small programs (called **microservices**). Each one does a single job and keeps its own data.

Service	Its job, in plain words	Keeps data in
product-service	The shop's catalogue. Add a product, or list all products.	MongoDB
order-service	Takes your order. First asks inventory 'do we have it?'. If yes, saves the order and shouts 'new order!' on Kafka.	MySQL (order_service)
inventory-service	The stock room. Answers 'is this item in stock?'	MySQL (inventory_service)

How the pieces fit together



In one sentence

Docker Desktop runs a small Linux computer inside your laptop. Kubernetes runs inside it and keeps your programs alive: if one crashes, Kubernetes starts it again. Harbor is your private 'app store' for the program images, also running inside Kubernetes.

What is running right now

Group	Running	Memory (approx.)
Your Java services	product-service, order-service, inventory-service	~1.3 GB
Their helpers	MongoDB, MySQL, Kafka broker, Zookeeper, Schema Registry	~1.5 GB
Harbor	core, portal, registry, database, redis, jobservice, nginx (7 parts)	~0.2 GB
Total		**~3 GB of the 7.8 GB the Docker VM has**

What is switched off (but kept)

These are still defined in Kubernetes with 0 copies running. They use no memory and can be switched back on with one command, or by adding the name to the list at the top of `k8s.ps1`.

Part	What it does	Why it is off
api-gateway	Single front door for all services, checks logins	Not needed for now
notification-service	Listens to Kafka and sends e-mails about orders	Not needed for now
keycloak + keycloak-mysql	Login server used by the gateway	Only the gateway needs it
grafana, prometheus, loki, tempo	Dashboards, metrics, logs and traces	Nice-to-have, heavy
kafka-ui	Web page to look inside Kafka	Nice-to-have
position-simulator, position-tracker, queue, webapp	Leftovers from an older course project (k8s-fleetman)	Not part of this project
frontend (Angular)	The web shop UI	**Deleted** at your request

2 Technical details

Tools, ports, databases and images

Technology stack

Area	What is used
Language / framework	Java 21, Spring Boot 3.2.4, Spring Web, Lombok
Data	Spring Data MongoDB (products); Spring Data JPA + MySQL 8.3 + Flyway migrations (orders, inventory)
Messaging	Spring Kafka + Confluent Kafka 7.5 with Avro messages and Schema Registry (topic: order-placed)
Resilience	Resilience4j circuit breaker on order -> inventory calls
API docs	springdoc-openapi: Swagger UI on every service
Monitoring hooks	Spring Actuator, Micrometer/Prometheus, Zipkin/Brave tracing, Loki log appender (their servers are off now - the services just skip sending)
Tests	Testcontainers, RestAssured, Spring Cloud Contract stubs
Build	Maven inside Docker (image maven:3.9-eclipse-temurin-21), runs on eclipse-temurin:21-jre
Platform	Docker Desktop 4.94, Kubernetes v1.36 (Kubeadm type, 1 node), Helm, Harbor

Ports and addresses inside the cluster

Name	Port	Notes
product-service	8080	REST: /api/product
order-service	8081	REST: /api/order, calls inventory-service:8082
inventory-service	8082	REST: /api/inventory?skuCode=..&quantity=..
mongodb	27017	user root / password, database product-service
mysql	3306	databases order_service and inventory_service
broker (Kafka)	9092 / 29092	with zookeeper:2181
schema-registry	8085	stores the Avro message format
Harbor	30002	reachable from Windows at http://localhost:30002

Where the program images come from

Every service runs from an **image** (a packed copy of the program). Ours are stored in Harbor:

```
localhost:30002/microservices/product-service:1.0
localhost:30002/microservices/order-service:1.0
localhost:30002/microservices/inventory-service:1.0
```

The helpers use public images: mongo:7.0.5, mysql:8.3.0, confluentinc/cp-kafka, cp-zookeeper and cp-schema-registry 7.5.0.

Where things live on your laptop

What	Where
Project source code	C:\Users\Lenovo\IdeaProjects\spring-boot-3-microservices-course
Kubernetes files (manifests)	...\k8s\manifests\infrastructure and ...\k8s\manifests\applications
The one script + your notes	C:\Users\Lenovo\Desktop\kubernetes\k8s.ps1
Docker build recipe	Dockerfile.service (written into the project folder by the script)

3 Kubernetes words, made simple

A small dictionary

Word	Meaning
Image	A frozen, ready-to-run package of a program. Like an installer that already contains everything.
Container	A running copy of an image, in its own little box.
Pod	The smallest thing Kubernetes runs: one (sometimes two) containers together.
Deployment	An instruction like 'always keep 1 copy of product-service running'. If the pod dies, a new one is started.
Replicas	How many copies to keep. 1 = on, 0 = switched off (but still remembered).
Service	A fixed name and port for a pod, e.g. inventory-service:8082. Pods come and go; the name stays.
Namespace	A folder for Kubernetes things. Ours: default (apps), harbor (Harbor), kube-system (Kubernetes itself).
Node	The computer that runs the pods. We have one: the Docker Desktop VM.
Manifest	A .yml text file that describes deployments and services. kubectl apply reads it.
kubectl	The command-line remote control for Kubernetes.
Registry	A store for images. Docker Hub is public; Harbor is your private one.
Helm	An installer for Kubernetes apps. Harbor was installed with it.
Port-forward	A temporary tunnel from your laptop's localhost to a service inside the cluster.
Kubeadm vs kind	Two kinds of Docker Desktop cluster. Kubeadm shares Docker's images and localhost; we need it.

4 How the setup was done

The full story, step by step

1 Check the laptop

Docker Desktop was already installed with Kubernetes turned on, plus Maven, IntelliJ, Eclipse and STS.

2 First deploy - straight from the course

All manifests from the repository were applied. The databases and Kafka started fine, but all 6 app pods kept crashing (CrashLoopBackOff).

3 Find the cause: wrong CPU type

The course's ready-made images (saiupadhyayula007/new-*) are built only for **arm64** (Apple Silicon). Your laptop is **x64**, so the programs could not start ('exec format error').

4 Build our own images

A small Dockerfile builds each service with Maven *inside Docker*, so nothing extra is needed on Windows. One library version (avro-maven-plugin) had to be pinned to 1.11.3 so the build works.

5 Add Harbor, a private registry

Harbor was installed with Helm into namespace 'harbor' (port 30002, no TLS, no virus scanner to save memory). A public project 'microservices' was created, images were pushed there, and the manifests were changed to use them. Docker Desktop was switched to the **Kubeadm** cluster type so 'localhost:30002' works for Docker and the cluster alike.

6 Make Harbor reachable from Windows

Docker Desktop forwards 'LoadBalancer' services to Windows, not 'NodePort' ones, so Harbor's service was changed to LoadBalancer on port 30002.

7 Clean up an old project

Pods from an older course project (k8s-fleetman) were still running in the same namespace and using memory.

8 Today: the laptop froze

The Docker VM had 7.8 GB of RAM, only **41 MB free**, no swap and a load of ~789. Kubernetes and Docker stopped answering. Docker Desktop was restarted twice; right after it came back, everything not needed was switched off.

9 Trim to what you asked for

Kept product, order, inventory + MongoDB, MySQL, Kafka, Zookeeper, Schema Registry + Harbor. Deleted the Angular frontend. Everything else: replicas = 0. Memory dropped from ~7.3 GB to ~3 GB, CPU from ~90% to ~4%.

10 One script instead of five

The five old scripts were merged into `k8s.ps1` and deleted. It was tested on your laptop and ran cleanly.

Why it froze, simply

Your laptop has 16 GB. Docker gets about half (7.8 GB). The full project - 6 services, Keycloak, Kafka, the Grafana stack and Harbor - needs more than that, so the VM ran out of memory and froze. Running only what you need right now keeps it at about 3 GB.

5 Human steps vs Claude's steps

Who does what - and how Claude did it

If you do it yourself: the human checklist

#	What a human does	How
1	Install Docker Desktop, turn on Kubernetes	Settings > Kubernetes > Enable, cluster type **Kubeadm**
2	Give Docker enough memory	Settings > Resources (or .wslconfig on WSL2). 8 GB is enough for the trimmed setup
3	Get the code	git clone into IdeaProjects
4	Install Helm, then Harbor	winget install Helm.Helm, then .\k8s.ps1 -Install does the rest
5	Build images and push to Harbor	.\k8s.ps1 -Build
6	Start only what you need	.\k8s.ps1
7	Open the apps in a browser	Port-forward (see section 6), then the links
8	When something is wrong	.\k8s.ps1 -Status, kubectl logs, or restart Docker Desktop

What Claude did in today's session

Step	What Claude did	Tool used
Read context	Read the project notes from earlier sessions to know the setup	Project notes
Get access	Asked to control Docker Desktop. Your Windows terminals are 'view-only' for Claude, so it asked you before turning on Docker Desktop's own terminal	Computer use + your approval
Diagnose	Ran kubectl and docker commands, read the answers from screenshots, found the VM was out of memory	Docker Desktop terminal
Recover	Restarted Docker Desktop; a waiting loop fired the clean-up commands the moment Kubernetes came back	Docker Desktop menu + PowerShell
Trim	Deleted frontend, scaled other apps to 0, turned Harbor back on, measured memory	kubectl / docker stats
Code reading	Downloaded your GitHub repo into its own cloud workspace to read controllers, ports and settings	Cloud workspace (git)
Files	Asked for access to Desktop\kubernetes, wrote k8s.ps1, asked before deleting the 5 old scripts	File bridge to your laptop
Test	Ran k8s.ps1 on your laptop and checked the output	Docker Desktop terminal
Remember	Updated the project notes so the next session knows all this	Project notes

How Claude works internally (behind the scenes)

Claude does not run *on* your laptop. It runs in a private cloud workspace and reaches your laptop through the Claude desktop app, which acts as a bridge. Through it, Claude has three kinds of hands:

Hand	What it can do	Limits
Computer use	Takes screenshots of your screen, clicks and types - like a person at the keyboard	Only in apps you approve. Terminals and browsers are view-only. A pop-up can block a button
File bridge	Reads and writes files in folders you grant, runs simple shell commands on those files	Only granted folders; deleting needs extra permission. Cannot reach Kubernetes from there
Cloud workspace	Its own Linux computer: downloads code from GitHub, writes scripts, makes this PDF	Cannot see your laptop or your cluster directly

The loop Claude repeats

1. Look	2. Think	3. Act	4. Check
Screenshot, or read a command's output	Compare with what should be there, choose the next command	Click or type one command (or a small batch)	Screenshot again; if wrong, try another way

Problems Claude hit, and the workaround

Problem	Workaround
Windows Terminal / cmd are view-only for Claude	Used Docker Desktop's built-in terminal (after asking you)
A 'Claude is using your computer' pop-up covered the terminal button	Waited for it to go away, then clicked
Pasted text with a newline did not run	Typed the command, then pressed Enter as a separate key
kubectl said 'TLS handshake timeout'	Checked memory inside the VM with wsl -d docker-desktop: almost 0 free
After a restart the cluster filled up again within a minute	Typed a loop that waits for Kubernetes to answer, then scales things down at once
No metrics-server for kubectl top	Used docker stats on the Kubernetes containers instead
The terminal panel is small	Wrote compact one-line output, scrolled and zoomed into screenshots

Safety and honesty notes

Claude asked before: controlling Docker Desktop, turning on the Docker terminal, opening your kubernetes folder and deleting the old scripts. It did not type passwords anywhere and did not change Windows settings. Steps 1-7 in section 4 happened in earlier sessions; this guide describes them from the project notes and the old scripts.

6 Using it every day

Start, browse, stop

After you switch the laptop on

Just start Docker Desktop and wait 2-3 minutes. Kubernetes remembers what should run, so only your trimmed set starts. You do **not** need to run anything.

The one script: k8s.ps1

Open a PowerShell in `Desktop\kubernetes` (or Docker Desktop's terminal) and run:

```
# only the wanted apps on, then show status
powershell -ExecutionPolicy Bypass -File .\k8s.ps1
# just show what runs + memory per container
powershell -ExecutionPolicy Bypass -File .\k8s.ps1 -Status
# rebuild the 3 services and push them to Harbor
powershell -ExecutionPolicy Bypass -File .\k8s.ps1 -Build
# fresh cluster: install Harbor, build, deploy
powershell -ExecutionPolicy Bypass -File .\k8s.ps1 -Install
```

Turning a service back on

Add its name to `$Apps` (or `$Infra`) at the top of k8s.ps1 and run the script again. Example: `$Apps = "product-service", "order-service", "inventory-service", "api-gateway"`. The gateway also needs keycloak and keycloak-mysql in `$Infra`.

Avoid this

Do not run `kubectl apply -f k8s\manifests\infrastructure` (a whole folder). It switches everything back on and the laptop can freeze again. Use k8s.ps1.

Open the services in your browser

Harbor works straight away. For the three services, first open the tunnels (one small window each, keep them open):

```
'product-service:8080','order-service:8081','inventory-service:8082' |
  % { $n,$p = $_ -split ':'
      start powershell "-NoExit kubectl port-forward svc/$n ${p}:$p" }
```

What	Link
Harbor UI (admin / Harbor12345)	http://localhost:30002
Harbor - your images	http://localhost:30002/harbor/projects
Harbor - API docs	http://localhost:30002/devcenter-api-2.0
Harbor - health	http://localhost:30002/api/v2.0/ping
Product - Swagger UI	http://localhost:8080/swagger-ui.html

What	Link
Product - list all products	http://localhost:8080/api/product
Product - health	http://localhost:8080/actuator/health
Order - Swagger UI (place orders here)	http://localhost:8081/swagger-ui.html
Order - health + circuit breaker	http://localhost:8081/actuator/health
Inventory - Swagger UI	http://localhost:8082/swagger-ui.html
Inventory - is it in stock?	http://localhost:8082/api/inventory?skuCode=iphone_15&quantity=1
Any service - API description (JSON)	http://localhost:808x/api-docs

Items already in stock (100 each): iphone_15, pixel_8, galaxy_24, oneplus_12.

Try a full order (in the Order Swagger UI)

```
POST /api/order
{
  "skuCode": "iphone_15",
  "price": 999,
  "quantity": 1,
  "userDetails": { "email": "you@example.com", "firstName": "Shikhin", "lastName": "Misra" }
}
```

Order-service asks inventory-service first. If the item is in stock, the order is saved in MySQL and an 'order-placed' message goes to Kafka. (Nobody reads that message right now, because notification-service is off.)

7 When something goes wrong

Symptoms and fixes

What you see	What it means	What to do
kubectl: 'TLS handshake timeout' or 'Unable to connect'	Kubernetes is overloaded or still starting	Wait 2 min. Still bad: Docker Desktop menu (bottom-left dots) > Restart, then run k8s.ps1
Docker Desktop shows RAM near 7.5 GB, everything slow	The VM is out of memory	Run k8s.ps1 (switches extras off). Check nothing extra was applied
Pod status CrashLoopBackOff	The program starts and crashes	kubectl logs deploy/<name> --previous
ImagePullBackOff / ErrImagePull	Cannot get the image	Is Harbor running? Open http://localhost:30002. Then k8s.ps1 -Build
exec format error	Image built for the wrong CPU (arm64)	Use the Harbor images (k8s.ps1 -Build), never the course's Docker Hub images
Browser: 'connection refused' on 8080/8081/8082	Port-forward window closed	Run the port-forward line again
Order fails with 'inventory' error	inventory-service down or SKU unknown	Check inventory pod; use a SKU from the list

Handy commands

```
kubectl get pods -A                              # everything that runs
kubectl get deploy                               # what is on (1) / off (0)
kubectl scale deploy api-gateway --replicas=1    # switch one thing on
kubectl scale deploy api-gateway --replicas=0    # ...and off again
kubectl logs deploy/order-service --tail=100     # last log lines
kubectl describe pod <pod-name>                  # why a pod is stuck
docker stats --no-stream                         # memory of each container
wsl -d docker-desktop free -m                    # free memory inside the Docker VM
```

Want more services later?

If you want to run everything again (gateway, Keycloak, Grafana...), raise Docker's memory first: create `C:\Users\Lenovo\.wslconfig` with `[wsl2]` and `memory=10GB`, then run `wsl --shutdown` and start Docker Desktop. Your laptop keeps 6 GB for Windows.

8 Quick reference card

Everything on one page

Thing	Value
Kubernetes context	docker-desktop (Kubeadm, 1 node)
Namespaces	default (apps), harbor (Harbor)
Harbor	http://localhost:30002 - admin / Harbor12345 - project 'microservices'
Images	localhost:30002/microservices/<service>:1.0
Services	product :8080 - order :8081 - inventory :8082
Databases	MongoDB :27017 (root/password) - MySQL :3306
Kafka	broker :9092, zookeeper :2181, schema-registry :8085, topic order-placed
Script	Desktop\kubernetes\k8s.ps1 (-Status / -Build / -Install)
Docker VM memory	7.8 GB (about 3 GB used with the trimmed setup)
Docker terminal	Docker Desktop, bottom-right >_ icon

Password note

Harbor12345 is Harbor's well-known default password. That is fine on a laptop that only you use, but change it before using Harbor anywhere else.

9 Rules for Claude (CLAUDE.md)

A template of standing instructions for future sessions

In the first session Claude had to make some decisions on its own, for example what to switch off when memory ran out. A **CLAUDE.md** file writes those decisions down once, so every future session follows the same rules without you repeating them. Think of it as a **note on the fridge for Claude**.

Where to put it

Where	What happens	When to use
Repo root: CLAUDE.md	Claude Code reads it automatically at the start of every session.	Main place for this project
A subfolder: CLAUDE.md	Applies only when working in that folder.	Rules for one service only
~/.claude/CLAUDE.md	Applies to all your projects on this laptop.	Personal habits
claude.ai Project instructions	Every chat in the Project sees the text.	Paste the same text there too

How to write good rules

- **Use numbers, not feelings.** "Under 2 GB available" is clear; "when resources are scarce" is not.
- **Say how to check the work.** For example: test with Postman via port-forward, or run `mvn verify`.
- **Split 'just do it' from 'ask me first'.** Anything hard to undo goes in the approval list.
- **Tune the 2 GB line.** The trimmed setup ran healthily at about 3 GB used of 7.8 GB (see the Quick reference card).
- **Keep it short and current.** Delete rules that no longer apply.

Coming back after months or years

Claude does not remember earlier chats by itself. What carries over is what is written down: this guide, the CLAUDE.md file in the repo, and the files in the claude.ai Project. Before you start again, check that the versions still match (Docker Desktop, Kubernetes, Spring Boot) and update the Environment section first.

The template

Copy this into a file named `CLAUDE.md` and change the values to match your setup.

```
# Project rules for Claude

## Environment
- Windows x64 laptop, 16 GB RAM; Docker Desktop Kubernetes
  (context `docker-desktop`), VM ~7.8 GB, no swap.
- Use `.\k8s.ps1` (Desktop\kubernetes) to deploy, build, or check
  status; don't `kubectl apply` whole manifest folders
  (it resets replicas to 1).
- Images must be built locally for x64 and pushed to Harbor
  (localhost:30002/microservices); Docker Hub images are arm64-only.

## Resource policy
- Before deploying, check available memory in the Docker VM.
- If available memory is under 2 GB, or the API server is slow or
  unresponsive: deploy backend only
  (product, order, inventory + mongodb, mysql, Kafka/schema-registry
  + Harbor).
  Do NOT deploy frontend, api-gateway, keycloak, or the observability
  stack (grafana, loki, prometheus, tempo).
- Backend-only is the default; the services are tested with Postman
  via `kubectl port-forward`.
- Ask me before enabling anything currently scaled to 0.

## Testing
- For every new or changed service feature, write integration tests
  with `@SpringBootTest`.
- Use Testcontainers for MongoDB, MySQL, and Kafka instead of the
  cluster's instances.
- Run `mvn verify` for the affected service before calling the work
  done; report failures, don't skip tests.

## Approval needed
- Deleting Kubernetes resources, Helm uninstalls, or deleting files:
  ask first.
- Scaling down, restarting pods, rebuilding images: go ahead, then
  tell me.
```

10 Claude Code setup

Rules, skills, helpers, automatic checks and connections

Since 9 October 2026 the project lives on GitHub as **Spring-microservices-on-kubernetes-setup-on-laptop** (private). Next to the code, the repo now contains everything Claude Code needs to work on it the same way every time - without you repeating the rules. Think of it as a **team handbook for Claude**, stored with the code.

The five pieces at a glance

Piece	Where in the repo	What it is	When it is used
CLAUDE.md	CLAUDE.md (repo root)	Short page of project facts, key commands and rules that always apply	Read automatically at the start of every session
Skills	.claude/skills/<name>/SKILL.md	Step-by-step recipes for jobs you repeat	Loaded only when the job comes up, or when you type /name
Subagents	.claude/agents/<name>.md	Specialist helpers with their own instructions and limited tools	Claude hands them a job (review, testing) and gets a report back
Hooks	.claude/settings.json + .claude/hooks/*.sh	Small scripts that run by themselves at fixed moments	Session start, before every shell command, after every file edit
MCP servers	.mcp.json	Connections to outside systems	When Claude needs to look at the cluster or GitHub

Where the files are now (GitHub repo)

Chapters 2 and 6 describe the first setup (`IdeaProjects\spring-boot-3-microservices-course` and `Desktop\kubernetes\k8s.ps1`). The repo reorganised it like this:

Folder	What is inside
services/	All Java code, one folder per service, plus the Angular frontend and the shared Dockerfile
k8s/infrastructure/core	MongoDB, MySQL, Kafka, Zookeeper, Schema Registry - the default helpers
k8s/infrastructure/extras	Keycloak, Kafka UI, Prometheus, Grafana, Loki, Tempo - only when there is memory
k8s/apps/overlays/local | full	local = backend only (default), full = everything. The image version numbers live here
argocd/	Ready-made files for the next step: automatic deployment with Argo CD
scripts/windows/	install-harbor.ps1, build-and-push.ps1, deploy.ps1 (they replace k8s.ps1)
docs/	This guide, setup notes, Claude Code setup notes

1. CLAUDE.md - the note on the fridge

Chapter 9 explained the idea and gave a template. The real file now exists in the repo root, adapted to the new folders and scripts. It is about 40 lines and has three parts:

Part	What it says (short version)
Project facts	Windows x64, 16 GB; Docker VM ~7.8 GB; cluster docker-desktop (kubeadm); where the Java code and k8s files are; Harbor address; ports of every service
Key commands	mvn verify for one service, docker build with services/Dockerfile, kubectl kustomize, build-and-push.ps1, deploy.ps1, kubectl logs
Rules	Backend only by default and never extras/full without asking - images always built for x64 and pushed to Harbor - never kubectl apply -f on a folder - every feature gets a @SpringBootTest + Testcontainers test and mvn verify must pass - ask before deleting, helm uninstall, force-push or Docker/WSL changes - scaling down, restarting and rebuilding are fine (tell afterwards) - change YAML in git, not by hand in the cluster - keep docs up to date

2. Skills - recipes Claude follows

Each skill is one folder with a SKILL.md. Only its one-line description is always in Claude's memory; the full recipe is loaded when needed. You can also start one yourself by typing its name with a slash.

Skill	Use it when	What Claude does	You could say
/new-service	A new microservice is needed	Creates the Maven module, code skeleton, integration test, Kubernetes files, image entry, script and README updates; then asks the reviewers	"Create a payment-service on port 8084 with MySQL"
/add-feature	Changing Java code in a service	Writes the test first, implements, runs mvn verify, asks code-reviewer, updates config/ConfigMaps	"Add a description field to products"
/release-version	A new version should run in the cluster	verify - build - push to Harbor - bump newTag in the overlay - commit - roll out - smoke test. **Only you can start it** (Claude will not run it by itself)	"/release-version order-service 1.1.0"
/deploy-local	Starting or redeploying on the laptop	Checks context, memory and Harbor; applies core + backend; waits until healthy; gives the browser links	"Bring the project up"
/troubleshoot-cluster	Something is broken or slow	Works through a symptom table (TLS timeout, CrashLoopBackOff, Pending, ImagePullBackOff, Harbor unreachable) with evidence	"Pods are stuck, check why"

3. Subagents - specialist helpers

A subagent is a second Claude with one job, its own instructions and only the tools it needs. It works in its own space and returns a short report, so the main conversation stays clean.

Helper	Its job	Can it change files?
code-reviewer	Reviews the latest Java changes: bugs, security, timeouts/circuit breaker, missing tests, config. Answers in Must fix / Should fix / Fine	No - read only
test-engineer	Writes and runs integration tests with Testcontainers (MySQL, MongoDB, Kafka), RestAssured and WireMock; reports pass/fail honestly	Only test files (src/test)
k8s-manifest-reviewer	Checks Kubernetes and Argo CD files: every kustomization renders, no arm64 or 'latest' images, memory limits, ports and labels match	No - read only, never changes the cluster

4. Hooks - automatic checks

Hooks are small bash scripts in `.claude/hooks/`. Claude Code runs them by itself; on Windows it uses Git Bash.

When	Script	What happens
Session starts	session-context.sh	Claude gets a snapshot: git branch and changed files, is the cluster reachable, pods not running, free memory in the Docker VM, is Harbor up
Before every shell command	guard-bash.sh	Checks the command against the rules (see the box below). It can block it or ask you first
After a .java file is edited	format-java.sh	Formats the file with Spotless (palantir style, set in the root pom.xml). Skips quietly if Maven is not installed
After a k8s/ or argocd/ YAML is edited	check-k8s.sh	Checks YAML syntax, no arm64 images, no 'latest' tags, and that every kustomization still renders. Problems go straight back to Claude to fix

What the guard does

Always blocked: the course's arm64 images - skipping tests in mvn - `kubectl apply -f` on a folder or with -R.
Asks you first: `kubectl delete` - `helm uninstall` - applying extras or overlays/full - force-push - `rm -r` - `wsl --shutdown`.
It only looks at the real command, so text inside a commit message or a document does not trigger it.

Changing the setup later

- New repeatable job - add a folder under `.claude/skills/`.
- New rule that is always true - add one line to CLAUDE.md. Keep it short; delete rules that no longer apply.
- After editing a hook, test it, e.g. `echo '{"tool_input":{"command":"kubectl delete pod x"}}' | bash .claude/hooks/guard-bash.sh`
- Personal settings go in `.claude/settings.local.json` or `CLAUDE.local.md` - both are ignored by git.

5. MCP servers - connections to outside systems

Connection	What Claude can do with it	Setting
kubernetes	Look at pods, logs, events and resources in the cluster. **Read only**: every change still goes through git and the scripts, where the rules and hooks apply	npx kubernetes-mcp-server, config in .claude/mcp/kubernetes.toml (read_only = true)
github	Issues, pull requests, branches and files of the repo	GitHub's remote MCP server; uses your token from the GITHUB_PAT variable

One-time setup on the laptop

#	Step	How
1	Install Node.js (needed for the kubernetes connection)	winget install OpenJS.NodeJS.LTS
2	Make a GitHub token	GitHub > Settings > Developer settings > Fine-grained token, only this repo: Contents, Issues, Pull requests (read/write)
3	Store it for your user (never in the repo)	setx GITHUB_PAT <token> - then open a new terminal
4	Start Claude Code in the repo folder	claude - approve the two project connections when asked; /mcp shows their status
5	Optional: Maven on PATH	winget install Apache.Maven - lets the format hook and mvn verify run outside Docker

How it all works together - an example

You say: "*Add a description field to products.*"

Step	What happens
Session start hook	Claude already knows the branch, that the cluster is up and how much memory is free.
add-feature skill	Claude follows the recipe: test first, then the code.
test-engineer	Writes the integration test with a real MongoDB in Testcontainers and runs mvn verify.
format hook	Every edited .java file is formatted automatically.
code-reviewer	Reviews the change; Claude fixes anything under 'Must fix'.
You: /release-version product-service 1.1.0	Build, push to Harbor, newTag bumped in git, rollout, smoke test.
Guard hook	If anything tries to apply the full stack or delete something, you are asked first.

Status (9 October 2026)

All files are in the repo (commit fa0bac9). The guard hook and the YAML check were tested in Claude's cloud workspace, and Claude Code loaded the skills there. Still to test on the laptop: the Java format hook, the two MCP connections, and mvn verify with Testcontainers. Known gap: inventory-service tests sit in a wrong folder with an illegal package name (java.com...), and product/notification have no tests yet - a good first job for test-engineer.